Chapter 28 worked solutions

Learning from Tables

Exercises 1–4 work through feature availability, row encoding, and probability scores. Exercises 9–11 develop evaluation plans for tabular models.

1. Decide when each feature exists

Imagine the service makes a prediction the moment a customer books a Friday appointment through the website. The website and Friday appointment are already recorded, even though the appointment itself is in the future. Use the information the service could have retrieved at the moment of booking.

- The **booking channel** is available. The service knows how the current booking was placed.
- The **scheduled date** is available. Use the date as entered at booking time. A later rescheduling would change the record after the prediction.
- The **cancellation fee actually charged** is unavailable. The actual charge is determined after booking, when the service handles the cancellation. Using it would leak information about the outcome into the model.
- The **customer's earlier completed bookings** are available if the service had recorded and could retrieve their completion by the time of the new booking. For example, if the customer made an earlier reservation for Thursday and placed the Friday booking on Wednesday, the Thursday reservation was still pending on Wednesday.

2. Separate new and missing categories

For the first row, fill in the blank number with the training median, 5, and set the numeric missing indicator to 1. The channel is web, so the web indicator is also 1. The six columns, in order, are filled numeric value, numeric missing indicator, mail, web, category missing indicator, and category unknown indicator.

- **Missing number, web.** The filled number and the two active indicators give $(5, 1, 0, 1, 0, 0)$.
- **2, missing channel.** Keep the number 2 and turn on the category missing indicator. The row is $(2, 0, 0, 0, 1, 0)$.
- **2, phone.** Keep the number 2 and turn on the category unknown indicator. Phone was supplied and falls outside the learned levels. The row is $(2, 0, 0, 0, 0, 1)$.

The last two rows have identical first four entries. The final two indicators let a model distinguish a blank channel from a supplied channel it has never seen.

3. Compute a context-weighted prediction

The first and third rows both have label 1, so their weights contribute to the positive-class prediction. The second row has label 0, so its contribution is zero. Multiply each label by its weight and add the results.

$$0.2(1) + 0.3(0) + 0.5(1) = 0.7.$$

If the second label changes from 0 to 1 and the weights stay fixed, its contribution goes from 0 to 0.3. The prediction increases by 0.3 to 1.0. All three rows now have label 1, so all of the weight goes to the positive class.

4. Distinguish probabilities from decisions

Both predictors assign the first case a probability below 0.5 and the second case a probability above 0.5. Their decisions are therefore $(0, 1)$, and both get two out of two cases correct.

The binary Brier score uses the original positive-class probabilities. For each case, subtract the observed label from the probability, square the result, and average the two squared errors.

$$Brier_1 = \frac{(0.2 - 0)^2 + (0.7 - 1)^2}{2} = \frac{0.04 + 0.09}{2} = 0.065.$$

$$Brier_2 = \frac{(0.4 - 0)^2 + (0.6 - 1)^2}{2} = \frac{0.16 + 0.16}{2} = 0.16.$$

The first predictor gives a probability closer to the observed label in both cases, so it has the lower Brier score. Thresholding discards those differences when it turns each probability into a class decision. Assessing calibration would require many held-out cases so that predicted probabilities could be compared with observed positive rates.

9. Split for the intended customer

A table of orders can contain several nearly identical rows from one customer. The same customer might repeatedly buy through the web and return similar products. Whether a model may learn from that person's earlier orders depends on who will be using the service when the model makes a prediction.

For **new customers**, assign whole customer IDs to the splits. One design uses 350 customers for training, 75 for validation, and 75 for test. Every order from a customer follows that customer's assignment. Check that the three ID sets are disjoint. The example specifies counts of customers because customers with more orders contribute more rows. Report the actual row counts and class counts too. Fit preprocessing and model parameters on the training customers. Choose settings on validation customers, then score the held-out customers. If the service predicts only at each new customer's first order, score only that customer's first eligible order. If it predicts on later orders from held-out customers too, report those orders separately or state the mix. Any history feature on an order must use information available at that order's prediction time.

For **later orders from existing customers**, split by global time cutoffs. For example, use orders placed through April for training, June for validation, and August for test. Define a test customer as existing if the customer had at least one order known before the test period. Score that person's August orders while retaining their earlier orders in the earlier splits. Define validation eligibility with its own earlier cutoff. This design allows an ID to occur in several splits, with each earlier record available before the later prediction. A separate per-customer cutoff could place orders from August in the training set while scoring June orders from other customers. Global cutoffs avoid that timing mismatch.

Keep the label observation window in mind. If a return label requires 30 days to mature, a recent order may be usable as history while its label is still unavailable for training. Leave May and July as gaps in the fitting and scoring windows so the preceding period's labels can mature before the next begins. Orders in a gap can supply history once they are observed. Freeze each fit and selection using only labels then known, and score test labels after their observation window ends. Track how many customers and orders qualify for the existing-customer protocol. A one-order customer whose only order occurred before the cutoff has no later order to score. The two protocols answer different deployment questions and can have different eligible populations.

10. Compare three methods fairly

Use one binary prediction task, such as whether an order will be returned. Compare logistic regression, a gradient-boosted decision tree ensemble, and a frozen TabPFN classification checkpoint. Fix the prediction time and the input fields. For instance, a model that predicts before shipping can see the sales channel and available order history. The date on which a later return was processed is part of the outcome history and is withheld.

Fix a training, validation, and test split that matches deployment, such as the grouped new-customer split. Draw nested subsets of 25, 50, 100, and 200 training customers, and use every eligible labeled order from the selected customers. Each larger subset contains the smaller one. At each size, give all three methods exactly the same labeled order rows. Report the actual row count because a customer can have several orders. Choose sizes that the chosen checkpoint can process and repeat the subset draws with recorded seeds. Use a common sampling rule that requires both classes in the smallest subset, based only on the training labels. A logistic model needs examples of both classes.

Fit the logistic weights and tree ensemble on the chosen labeled rows. For logistic regression, learn missing-value fills, category levels, and any scaling from that subset. The tree ensemble may use its own handling of categorical or missing values under a fixed policy. Any learned encodings come from the same subset. The frozen in-context model gets those rows and their labels as context and the query's available features without its label. Its pretrained weights supply prior information, and the task's labeled data budget is the same subset. Fix the checkpoint revision, feature types, and context construction rule.

Use the separate validation set to select regularization, tree settings, and any allowed choices for the in-context model. Set a comparable tuning budget and disclose how many validation labels each method uses. Fix a threshold on validation if evaluating decisions. Learn no preprocessing or model choices from the test rows. Reuse the same test set at every size. Report the mean and variation across subset draws for a decision score such as accuracy and a probability score such as the Brier score. Plot or tabulate the results against labeled training size. With the test examples fixed, changes along a curve reflect the amount and composition of the available labeled training data. Record fit and prediction costs separately if compute matters, since loading a checkpoint and supplying context has different costs from fitting new weights.

11. Investigate unreliable probabilities

Suppose predictions in the 0.7–0.8 bin average 0.74, but the observed positive rate is 0.50. The gap is 0.24 in this bin. First count the orders, distinct customers, and positive labels. Two positives among four orders and 200 among 400 both give 0.50, with different precision. Confirm that the outcome window is complete. If recent returns are still arriving, a premature count can make the positive rate look too low.

Check which customers appear in the latest orders, training, and earlier validation. Report overlap counts and the gap separately for new and existing customers when there are enough cases. Many repeated orders from a few customers provide less independent information than orders from many customers. If estimating uncertainty, keep each customer's orders together when resampling. In a time-ordered setting also inspect how the gap changes by week. A single interval for the latest bin can conceal a short change in the customer or order mix.

Check input distributions across the periods, using features as they existed at prediction time. Compare channel shares, missing and unseen-category rates, numerical feature summaries, and the fraction of new customers. Also compare overall positive rates after labels mature. Feature drift is a change in the inputs the model receives. A change in the bin gap can also come from changes in outcomes at similar inputs. Rerun the same bin definitions on predictions from the same frozen model in an earlier validation period. Compare each bin's mean probability with its observed rate and report counts. Use the same boundary rule, such as including 0.7 and excluding 0.8.

Treat the latest data examined above as a diagnostic period. If adjusting probabilities, fit a recalibration map on a separate labeled period. For example, fit a logistic mapping of the model's log-odds to the observed labels, using a fixed clipping rule for predictions of 0 or 1. The log-odds for probability p are $\ln(p/(1-p))$. Freeze the map, bin definitions, and scoring rules, then evaluate the original and adjusted probabilities on the same later, untouched test orders. Report bin counts, reliability and Brier score for both. This checks whether the adjustment prompted by the diagnostic gap improves future probability estimates.